

---

# Storing RDF-based Knowledge Graphs

**Ernesto Jiménez-Ruiz**

Senior Lecturer in Artificial Intelligence

---

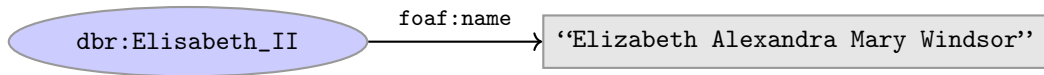
# RDF serialisations

# RDF Serialisations

There are many serialisations for the RDF data model:

- RDF/XML the W3C standard
- **Turtle**
- JSON-LD
- N3
- TriG

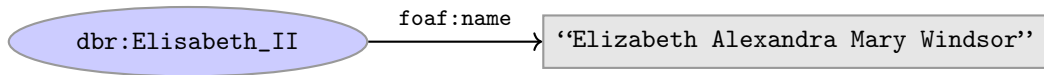
# RDF/XML Serialisation



Machine readable:

```
<?xml version="1.0"?>
<rdf:RDF xmlns:dbp="http://dbpedia.org/resource/"
         xmlns:foaf="http://xmlns.com/foaf/0.1/"
         xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">
  <rdf:Description rdf:about="http://dbpedia.org/resource/Elisabeth_II">
    <foaf:name>Elizabeth Alexandra Mary Windsor</foaf:name>
  </rdf:Description>
</rdf:RDF>
```

# Turtle Serialisation



Human readable/writable

```
@prefix dbr: <http://dbpedia.org/resource/> .
```

```
@prefix foaf: <http://xmlns.com/foaf/0.1/> .
```

```
dbr:Elisabeth_II foaf:name "Elizabeth Alexandra Mary Windsor" .
```

# Turtle: URI references and triples

**Full URIs are surrounded by < and >:**

`<http://dbpedia.org/resource/London>`

# Turtle: URI references and triples

**Full URIs are surrounded by < and >:**

```
<http://dbpedia.org/resource/London>
```

**Statements are triples terminated by a period '.':**

```
<http://dbpedia.org/resource/London>  
  <http://www.w3.org/1999/02/22-rdf-syntax-ns#type>  
    <http://dbpedia.org/ontology/City> .
```

# Turtle: URI references and triples

**Full URIs are surrounded by < and >:**

```
<http://dbpedia.org/resource/London>
```

**Statements are triples terminated by a period '':**

```
<http://dbpedia.org/resource/London>  
  <http://www.w3.org/1999/02/22-rdf-syntax-ns#type>  
    <http://dbpedia.org/ontology/City> .
```

**Use 'a' to abbreviate `rdf:type`:**



# Turtle: Namespaces

QNames/CURIs are written without any special characters (*e.g.*, < and >).

# Turtle: Namespaces

QNames/CURIs are written without any special characters (*e.g.*, < and >).

**Namespace prefixes are declared with @prefix:**

```
@prefix dbr: <http://dbpedia.org/resource/> .
```

```
dbr:London a <http://dbpedia.org/ontology/City> .
```

# Turtle: Namespaces

QNames/CURIs are written without any special characters (*e.g.*, < and >).

**Namespace prefixes are declared with @prefix:**

```
@prefix dbr: <http://dbpedia.org/resource/> .
```

```
dbr:London a <http://dbpedia.org/ontology/City> .
```

**A default namespace may be declared:**

```
@prefix dbr: <http://dbpedia.org/resource/> .
```

```
@prefix : <http://dbpedia.org/ontology/> .
```

```
dbr:London a :City .
```

# Turtle: Literals

## Literal values are enclosed in double quotes:

```
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
```

```
@prefix dbr: <http://dbpedia.org/resource/> .
```

```
@prefix : <http://dbpedia.org/ontology/> .
```

```
dbr:London rdfs:label "City of London" .
```

# Turtle: Literals

## Literal values are enclosed in double quotes:

```
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
```

```
@prefix dbr: <http://dbpedia.org/resource/> .
```

```
@prefix : <http://dbpedia.org/ontology/> .
```

```
dbr:London rdfs:label "City of London" .
```

## Possibly with type or language information:

```
dbr:London rdfs:label "Londres"@es .
```

```
dbr:London :population "9,304,000"^^xsd:integer .
```

# Turtle: Literals

## Literal values are enclosed in double quotes:

```
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
```

```
@prefix dbr: <http://dbpedia.org/resource/> .
```

```
@prefix : <http://dbpedia.org/ontology/> .
```

```
dbr:London rdfs:label "City of London" .
```

## Possibly with type or language information:

```
dbr:London rdfs:label "Londres"@es .
```

```
dbr:London :population "9,304,000"^^xsd:integer .
```

## Numbers and booleans may be written without quotes:

```
dbr:London :population 9,304,000 .
```

```
dbr:London :isCapital true .
```

# Turtle: Statements sharing elements (i)

## Instead of:

```
dbr:London rdf:type dbo:City .  
dbr:London rdfs:label "London" .  
dbr:London :population 9,304,000 .
```

# Turtle: Statements sharing elements (i)

## Instead of:

```
dbr:London rdf:type dbo:City .  
dbr:London rdfs:label "London" .  
dbr:London :population 9,304,000 .
```

## ... statements may share a subject with ‘;’:

```
dbr:London rdf:type dbo:City ;  
            rdfs:label "London" ;  
            :population 9,304,000 .
```



## Turtle: Statements sharing elements (ii)

### Instead of:

```
dbr:London rdfs:label "London"@en .  
dbr:London rdfs:label "Londres"@es .  
dbr:London rdfs:label "Londra"@it .
```

## Turtle: Statements sharing elements (ii)

### Instead of:

```
dbr:London rdfs:label "London"@en .  
dbr:London rdfs:label "Londres"@es .  
dbr:London rdfs:label "Londra"@it .
```

### ...statements may share subject and predicate with ‘,’:

```
dbr:London rdfs:label "London"@en ,  
                    "Londres"@es ,  
                    "Londra"@it .
```

## Turtle: Blank nodes

Blank nodes are designated with underscores or [...].

**England has a capital with population 9,304,000:**

```
dbr:England :capital _:someplace .  
_:someplace :population 9,304,000 .
```

## Turtle: Blank nodes

Blank nodes are designated with underscores or [...].

**England has a capital with population 9,304,000:**

```
dbr:England :capital _:someplace .  
_:someplace :population 9,304,000 .
```

**There is a city with official name London:**

```
[] a :City ;  
    :officialName "London" .
```

## Turtle: Blank nodes

Blank nodes are designated with underscores or [...].

**England has a capital with population 9,304,000:**

```
dbr:England :capital _:someplace .  
_:someplace :population 9,304,000 .
```

**There is a city with official name London:**

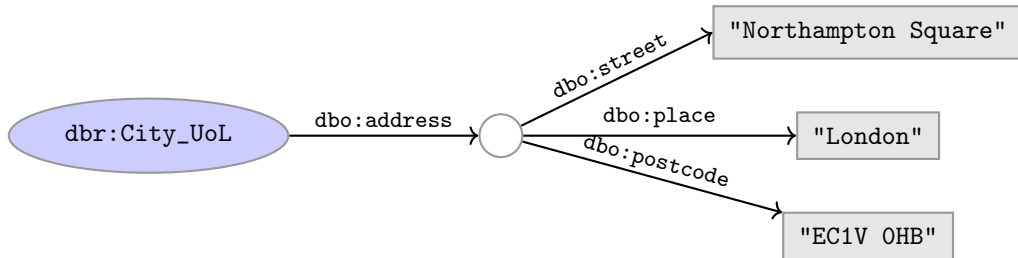
```
[] a :City ;  
    :officialName "London" .
```

**City has address Northampton Square, EC1V 0HB:**

```
:City_UoL :address [ :street "Northampton Square" ;  
                      :postcode "EC1V 0HB" ] .
```

## Turtle: Blank nodes (Question)

The blank node here:

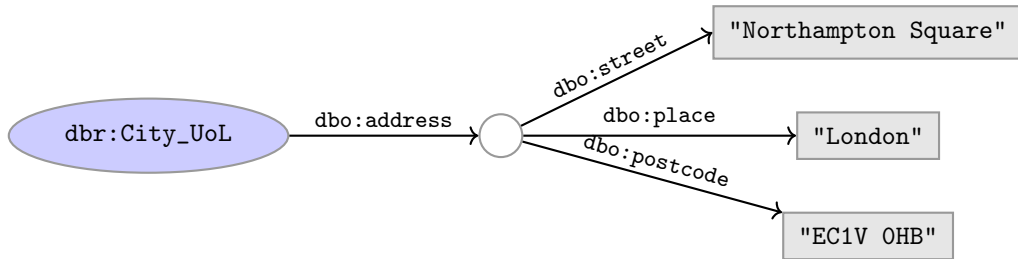


has no 'name' or 'id'.

Why does Turtle use 'blank node identifiers' like `_:someplace`?

## Turtle: Blank nodes (Question)

The blank node here:



has no 'name' or 'id'.

Why does Turtle use 'blank node identifiers' like `_:someplace`?

Answer: makes it possible to use same node in several triples.

# Turtle: Merging RDF files (i)

Merging two RDF files containing named blank nodes

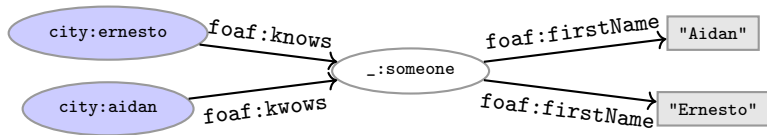
## File 1

```
city:ernesto foaf:knows _:someone .  
_:someone foaf:firstName "Aidan" .
```

## File 2

```
city:aidan foaf:knows _:someone .  
_:someone foaf:firstName "Ernesto" .
```

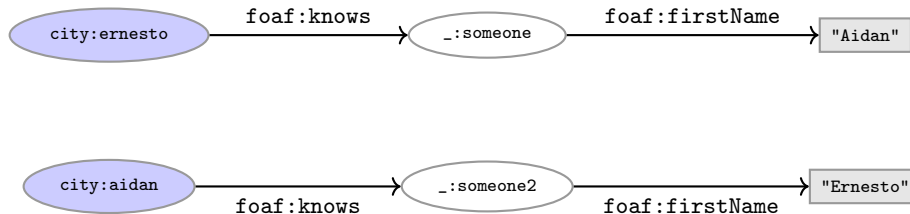
gives the RDF graph:





## Turtle: Merging RDF files (i)

Solution: Renaming `_:someone` to `_:someone2` in File 2.



## **Turtle: complex statements**

We can use triples to form complex statements, e.g.:

# Turtle: complex statements

We can use triples to form complex statements, e.g.:

## Data structures

```
:INM373 :hasLecturer
  [ rdf:first :epriego ;
    rdf:rest [ rdf:first :ernesto ;
               rdf:rest [ rdf:first :carlos ;
                           rdf:rest [ rdf:first :andy ;
                                       rdf:rest [ rdf:first :nikos ;
                                                  rdf:rest rdf:nil ]
                                     ]
               ]
    ] ] ] .
```

# Turtle: complex statements

We can use triples to form complex statements, e.g.:

## Data structures

```
:INM373 :hasLecturer
  [ rdf:first :epriego ;
    rdf:rest [ rdf:first :ernesto ;
               rdf:rest [ rdf:first :carlos ;
                           rdf:rest [ rdf:first :andy ;
                                       rdf:rest [ rdf:first :nikos ;
                                                  rdf:rest rdf:nil ]
                                     ]
               ]
    ] ] ] .
```

## Turtle shorthand for lists

```
:INM373 :hasLecturer (:epriego :ernesto :carlos :andy :nikos) .
```

# Turtle: TriG to support named graphs

- TriG RDF syntax extends Turtle with support for named graphs.

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix : <http://www.example.org/university/london/city#> .

:G1 {
  :INM373 rdf:type :Module .
  :INM373 :hasLecturer ( :epriego :ernesto :carlos :andy :nikos ) .
  :INM373 :module_type "Core" }
:G2 {
  :IN3067-INM713 rdf:type :Module .
  :IN3067-INM713 :hasLecturer ( :ernesto ) .
  :IN3067-INM713 :module_type "Elective" }
```

# Turtle: Other things

## Use '#' to comment:

```
# This is a comment.
```

```
dbr:London a dbo:City . # This is another comment.
```

## Turtle: Other things

### Use '#' to comment:

```
# This is a comment.  
dbr:London a dbo:City . # This is another comment.
```

### Use '\' to escape special characters:

```
:someGuy :foaf:name "James \"Mr. Man\" Olson" .
```

Turtle specification: <http://www.w3.org/TR/turtle/>.

---

# Storing RDF Graph Models



## How to store (RDF) Graph models?

- A file (*e.g.*, RDF/XML or Turtle serializations)
- Relational model
  - Single table with 3 columns (source, edge, target)
  - Property tables (one table per type of entity, *e.g.*, Person, Module)
  - Binary tables (one table per relationship, *e.g.*, source, target)

M. Wylot and others. RDF Data Storage and Query Processing Schemes. ACM Computing Surveys 2018

## How to store (RDF) Graph models?

- A file (*e.g.*, RDF/XML or Turtle serializations)
- Relational model
  - Single table with 3 columns (source, edge, target)
  - Property tables (one table per type of entity, *e.g.*, Person, Module)
  - Binary tables (one table per relationship, *e.g.*, source, target)
- **Graph-based databases** (NoSQL)
  - During Session 7 (Week 8) we will compare some cases.

M. Wylot and others. RDF Data Storage and Query Processing Schemes. ACM Computing Surveys 2018

## NoSQL (Not only SQL) databases (i)

- **No strict definition** of NoSQL.
- They typically have a **denormalised** data model.
  - Data has some duplication.
  - Query performance is increased.
  - Writing and updates are less efficient.
- **No integrity constraints (*i.e.*, more flexibility).**
- Consistency is checked afterwards (“eventually consistent” models).

## NoSQL (Not only SQL) databases (ii)

Types of NoSQL databases:

- Key-Value pair-based databases (*e.g.*, ArangoDB)
- Column-oriented databases (*e.g.*, MonetDB)
- Document-oriented databases (*e.g.*, MongoDB, ArangoDB)

(\*) Object-oriented databases can also be seen as a type of NoSQL but they include data integrity.

(\*\*) RDFLib and Apache Jena are libraries to deal with RDF graphs.

## NoSQL (Not only SQL) databases (ii)

Types of NoSQL databases:

- Key-Value pair-based databases (*e.g.*, ArangoDB)
- Column-oriented databases (*e.g.*, MonetDB)
- Document-oriented databases (*e.g.*, MongoDB, ArangoDB)
- **Graph databases** (*e.g.*, Neo4j, Virtuoso, Amazon Neptune, Oracle)
  - **RDF Triplestores** (*e.g.*, RDFox, Apache Jena TBD, GraphDB)
  - Most Graph databases are also suitable as RDF triplestores.

(\*) Object-oriented databases can also be seen as a type of NoSQL but they include data integrity.

(\*\*) RDFLib and Apache Jena are libraries to deal with RDF graphs.

# Contact

- **Ernesto Jiménez-Ruiz**
- City St George's, University of London
- Contact:
  - `ernesto.jimenez-ruiz@citystgeorges.ac.uk`
  - `https://ernestojimenezruiz.github.io/`